

Rest API expand mechanism

#dotnet

#csharp

Theme	Expand mechanism in Rest API
-------	------------------------------

Platform	Blog
----------	----------------------

Release date	September 30, 2026
--------------	--------------------

Outline

1. Introduction

- ~~Introduce a practical example~~
- ~~Explain the limitations of separate HTTP requests and always including related data.~~
- ~~Introduce resource expansion~~

2. What Is Resource Expansion?

- ~~Explain how expansion works.~~
- ~~Show a request such as `GET /orders/42?expand=customer,items`.~~
- ~~Compare responses with and without expansion.~~
- ~~Clarify the difference between fewer HTTP requests and fewer database queries.~~

3. Implementing Resource Expansion in ASP.NET Core

- ~~Define the entities, response DTOs, and baseline endpoint.~~
- ~~Parse and validate the `expand` parameter using an allowlist.~~
- ~~Load the requested relationships with EF Core.~~
- ~~Build the response and define how unrequested relationships are represented.~~
- ~~Handle unsupported expansions~~
- ~~Limit expansion depth and related collection sizes.~~
- ~~Show the complete implementation with example requests and responses.~~

4. Performance: Expanded Requests vs. Separate Requests

- ~~Compare approaches that return equivalent data.~~
- ~~Describe the dataset, environment, and request execution strategy.~~
- ~~Measure end-to-end latency, response size, and database query count.~~
- ~~Examine how larger related collections affect performance.~~
- ~~Explain when expansion improves performance and when it adds overhead.~~

5. Summary

- ~~Summarize the advantages and disadvantages in a table.~~
- ~~Explain when optional expansion is useful.~~
- ~~Identify cases better served by separate or dedicated endpoints.~~
- Discuss the maintenance cost of supporting multiple response shapes.

Draft

Recently, my team decided to have one endpoint that could return related data for the requested entity. Our standard approach was to use separate endpoints, but we wanted to avoid multiple HTTP round trips.

Inspired by how Stripe and Atlassian use expansion in their APIs, we decided to introduce an expand mechanism of our own.

What is resource expansion?

Resource expansion is a mechanism that includes related data in the main resource's response when the client explicitly requests it.

<diagram>

Part of application you are developing needs to show membership for particular user.

`GET /memberships/42` will provide you response like this:

<code snippet of response>

But the client also needs additional membership details:

- The type of program the user paid for.
- The amount paid.
- The membership duration.
- Extra features, such as access to the sauna or pool.

Following approach from above we could get that data from another endpoint and then map both response into viewmodel and sent it to the client.

Or to request it along side main endpoint with `expand` query parameter. query parameter.

`GET /memberships/42?expand=details`

<code snippet of response>

In single HTTP request, we get all data that is required by the client. What you need to know is that doesn't mean less database queries. It all **depends on how backend is structured**.

Implementation in .NET

First, let's lay the foundation:

- Entites
- Request/Response
- Endpoint

Entities

<code snippet for entitles>

The relationships are straightforward: a member can have multiple memberships, a membership can have multiple visits, and each visit belongs to one club.

Request and response object

Request

<code snippet>

The `Expand` property specifies which supported relationships the client wants to include in the response.

Response

<code snippet>

The response contains the basic membership fields and optional `Member` and `Visits` properties. These properties are populated only when the corresponding relationships are requested.

Endpoint

Next, we define the endpoint that returns a membership by its ID:

<endpoint>

With the foundation in place, on to expansion.

Expand parameter

The endpoint supports two relationships, which clients can request in three combinations:

```
{ } JSON
```

```
GET /memberships/42?expand=member
GET /memberships/42?expand=visits
GET /memberships/42?expand=member,visits
```

Since `expand` is a query parameter, clients can pass any value. Therefore allowed relationships and input validation is must.

<code snippet whitelist>

Validator have two steps:

1. **Parsing and normalization:** Split the `Expand` value into individual relationship names, trim whitespace, ignore empty entries, and remove duplicates using a case-insensitive comparison.
2. **Validation:** Check the parsed names against the whitelist to ensure every requested relationship is supported.

<code snippet for parsing>

I've used FluentValidation for the validation step

<code snippet validator>

When validation is passed we can build the query:

<code snippet with query>

The idea is simple: check which relationships were requested and add the corresponding `Include` calls.

<code snippet for query execution>

Finally, fetched data is mapped to the response object.

<code snippet for response mapping>

In this response contract, unrequested relationships are represented by `null`. A requested collection with no items is represented by `[]`.

With both `member` and `visits` requested, the response looks like this:

<code snippet for expanded response>

Performance

Expansion reduces HTTP calls, but does it improve performance?

So, I tested two set of API calls that will return same result:

Separate request approach:

Text

```
GET /memberships/42
GET /members/7
GET /memberships/42/visits
```

Expand approach:

Text

```
GET /memberships/42?expand=member,visits
```

Test setup

The tests run on my laptop with the following specifications:

<CPU, RAM, and operating system>

The application and database run in Docker, using this setup:

<.NET and EF Core versions, database provider and version, Docker resource limits>

The benchmark client runs <on the host / inside Docker>. Both approaches use the same caching and response-compression settings.

What is measured?

For separate requests, it's measured the total time from sending the first request until all three response bodies have been received.

For expansion, the time until the complete expanded response has been received.

quote: Before collecting results, warm-up iterations are run.

Each scenario is then repeated times with concurrent clients.

The comparison covers:

- **Median latency:** The middle response time across all measured operations.
- **P95 latency:** The time within which 95% of operations complete.
- **Response size:** The total response-body bytes received per operation, summed across separate requests.
- **Database query count:** The total SQL commands executed per operation.

Results

<results>

<Explain which approach was faster, by how much, and whether response size or database query count changed.>

Fewer HTTP requests do not automatically mean fewer database queries. The expanded endpoint still needs to retrieve all requested data, and its query strategy determines how much work reaches the database.

So the verdict is:

Expansion gives clients a convenient way to fetch related data together. Whether it improves performance depends on the network, the database queries, and the amount of data requested.

When to use resource expansion?

Honestly, I don't have straightforward answer to. This approach comes with trade-offs:

Advantages	Disadvantages
Fewer HTTP round trips to retrieve related resources.	Expanded responses can become large and expensive to generate.
Clients choose which supported relationships to include.	Different expansion combinations require additional validation and testing.
Less client-side work combining separate responses.	Database queries can become more complex as relationships are added.
One endpoint supports different data needs.	More response variations need to be documented and maintained.

But what I think can be sum up is: keep expansions focused on client needs and measure impact before assuming fewer requests mean better performance.